Blackboard Exercise 3: Worked solutions

Algorithms, Programming, and Data Representation · Module 92313 · Blackboard Exercise 3

From Fundamentals to AI Apocalypse · personal study guide

Contents

Exercise 3.1: Memory chip architecture 2

Exercise 3.2: Memory modules architecture 3

Exercise 3.3: Byte-addressable memory modules 3

Exercise 3.4: Endianness bytes 4

Answers 5

Made by Thanate Amornjiraporn (Focus)

Work through memory size, addressing, module construction, interleaving and byte order. For address calculations, state whether an address selects a word or a byte.

Exercise 3.1: Memory chip architecture

Connected topic: Addressing and capacity (§5.2)

3.1.1 Memory size in bits

Try first: For a 4G × 32-bit memory, find the total capacity in bits and write it as 2^x.

Answers: see "Exercise 3.1: Memory chip architecture" under Answers at the end.

3.1.2 Word-addressable memory

Try first: Find the address width when each address selects one 32-bit word.

Answers: see "Exercise 3.1: Memory chip architecture" under Answers at the end.

3.1.3 Byte-addressable memory

Try first: Find the address width when each address selects one byte.

Answers: see "Exercise 3.1: Memory chip architecture" under Answers at the end.

3.1.4 Byte address range

Try first: Give the lowest and highest byte addresses in hexadecimal and the number of bits needed to represent them.

Answers: see "Exercise 3.1: Memory chip architecture" under Answers at the end.

Made by Thanato Amornjirapon (Focus)

Exercise 3.2: Memory modules architecture

Connected topic: Memory chips and modules (§5.4) · Memory interleaving (§5.5)

3.2.1 Module configuration

Try first: Complete the chip and address-width specifications for a word-addressable 1G × 32-bit memory built from 256M × 4-bit chips.

Answers: see "Exercise 3.2: Memory modules architecture" under Answers at the end.

3.2.2 Word 14 across four modules

Try first: Locate word address 14 under high-order and low-order interleaving, numbering modules from zero.

Answers: see "Exercise 3.2: Memory modules architecture" under Answers at the end.

3.2.3 Word 300,000,000

Try first: Find the module and local word address for word address 300,000,000 under both interleaving schemes.

Answers: see "Exercise 3.2: Memory modules architecture" under Answers at the end.

Exercise 3.3: Byte-addressable memory modules

Connected topic: Addressing and capacity (§5.2) · Memory interleaving (§5.5)

3.3.1 Full-memory address width

Try first: For the same 1G × 32-bit memory, find the number of address bits when it is byte-addressable.

Answers: see "Exercise 3.3: Byte-addressable memory modules" under Answers at the end.

3.3.2 Word 14 after byte-addressing

Try first: Locate word 14 under both interleaving schemes. The memory modules still store complete 32-bit words.

Answers: see "Exercise 3.3: Byte-addressable memory modules" under Answers at the end.

Made by Thanate Amornjitaporn (Focus)

Exercise 3.4: Endianness bytes

Connected topic: Endianness and aligned access (§5.3)

3.4.1 Define endianness

Try first: State the definition of endianness in one sentence.

Answers: see "Exercise 3.4: Endianness bytes" under Answers at the end.

3.4.2 Transfer between different byte orders

Try first: Explain what must happen when data moves from little-endian memory to big-endian memory.

Answers: see "Exercise 3.4: Endianness bytes" under Answers at the end.

3.4.3 Transfer 0x12345678 unchanged

Try first: A 32-bit integer 0x12345678 is stored little-endian and copied unchanged into big-endian memory. Find the value read there and decide whether the original can be recovered without byte-order information.

Answers: see "Exercise 3.4: Endianness bytes" under Answers at the end.

Made by Thanate Amornjiaporn (Focus)

Answers

Exercise 3.1: Memory chip architecture

3.1.1 Memory size in bits · worked solution

1. In the notation 4G × 32, there are 4G addressable words and 32 bits per word. For this exercise, G means 2^{30}, so 4G = 2^{32} words.
2. Multiply words by bits per word: $2^{32} \times 32 = 2^{32} \times 2^5 = 2^{37}$ bits. Thus x = 37.
3. The check is dimensional: word count × bits/word gives bits. The 32-bit width adds five powers of two to the 4G word count.

Exercise 3.1: Memory chip architecture

3.1.2 Word-addressable memory · worked solution

1. The memory contains 4G = 2^{32} words. Word-addressing assigns one distinct address to each word.
2. The number of address bits is $\log_2(2^{32}) = 32$ bits.
3. A 32-bit address has 2^{32} patterns, exactly enough for all word addresses from 0 through $2^{32}-1$.

Exercise 3.1: Memory chip architecture

3.1.3 Byte-addressable memory · worked solution

1. Each 32-bit word contains 4 bytes. The number of bytes is 2^{32} words × 4 bytes/word = 2^{34} bytes.
2. Addressing 2^{34} individual bytes requires 34 address bits.
3. This is two more bits than word addressing because each word contains 2^2 byte locations.

Exercise 3.1: Memory chip architecture

3.1.4 Byte address range · worked solution

1. There are 2^{34} byte addresses, numbered consecutively from zero. The highest address is therefore the count minus one.
2. The lowest address is 0x000000000. The highest is $2^{34}-1$ = 0x3FFFFFFFF. The address width is 34 bits.
3. Check that the next address, 0x400000000, is 2^{34} and is outside the memory. Leading zeroes in the lowest address are only padding.

Exercise 3.2: Memory modules architecture

3.2.1 Module configuration · worked solution

1. A chip supplies 4 bits per word, so eight chips in parallel supply the required 32-bit word. Each module then has 256M = 2^{28} words. Four such modules provide 1G = 2^{30} words.
2. The full system uses $8 \times 4 = 32$ chips. A module needs $\log_2(2^{28}) = 28$ row-address bits; the full memory needs $\log_2(2^{30}) = 30$ word-address bits.
3. The configuration is:

Specification	Quantity
RAM chips per memory module	8
Memory modules	4
RAM chips for full memory	32
Address bits per module	28
Address bits for full memory	30

4.
5. Check capacity and width separately: 4 modules × 256M words = 1G words; 8 chips × 4 bits = 32 bits per word.

Exercise 3.2: Memory modules architecture

3.2.2 Word 14 across four modules · worked solution

1. There are four modules, so two address bits select a module. Each module holds 2^{28} words. High-order interleaving uses the most significant address bits for the module; low-order interleaving uses the least significant two bits.
2. High-order: module = floor($14/2^{28}$) = 0; local word address = 14. Low-order: module = 14 mod 4 = 2; local word address = floor(14/4) = 3.
3. Check by reconstructing the addresses. High-order module 0 starts at word 0, so local 14 is global 14. Low-order module 2 stores words 2, 6, 10, 14, …; its fourth row is local address 3.

Exercise 3.2: Memory modules architecture

3.2.3 Word 300,000,000 · worked solution

1. For high-order interleaving, divide by the module capacity 2^{28} = 268,435,456. For low-order interleaving, divide by four modules.
2. High-order: 300,000,000 = 1 × 268,435,456 + 31,564,544, so the location is module 1, local word address 31,564,544. Low-order: 300,000,000 mod 4 = 0 and floor(300,000,000/4) = 75,000,000, so the location is module 0, local word address 75,000,000.
3. Reconstruct: high-order 1×2^{28} + 31,564,544 = 300,000,000; low-order 4 × 75,000,000 + 0 = 300,000,000.

Exercise 3.3: Byte-addressable memory modules

3.3.1 Full-memory address width · worked solution

1. The capacity is 1G words × 32 bits/word. Each word is four bytes, so there are 1G × 4 = 4G = 2^{32} bytes.
2. The full byte-addressable memory therefore needs $\log_2(2^{32}) = 32$ address bits.
3. Check against word-addressing: 1G words require 30 bits; the two byte-within-word bits raise the byte address to 32 bits.

Exercise 3.3: Byte-addressable memory modules

3.3.2 Word 14 after byte-addressing · worked solution

1. Word index 14 begins at byte address 14 × 4 = 56. Because each module stores a complete 32-bit word, select the module using the word index; the two lowest byte-address bits identify the byte within that word.
2. High-order interleaving: word 14 is in module 0 at local word row 14, beginning at local byte address 56. Low-order interleaving: module = 14 mod 4 = 2 and local word row = floor(14/4) = 3; the word begins at local byte address 3 × 4 = 12. Its global byte address is still 56.
3. The low-order module bits come from the word address after removing the two byte-offset bits. Reconstruct the global word index: 4 × 3 + 2 = 14. The first byte of that word is 56.

Exercise 3.4: Endianness bytes

3.4.1 Define endianness · worked solution

1. A multi-byte value occupies several byte addresses, so the system needs a rule for which byte comes first.
2. Endianness is the order in which a multi-byte value's bytes are stored in memory: little-endian places the least significant byte at the lowest address, while big-endian places the most significant byte there.
3. This rule affects multi-byte values; a single-byte value has no byte-order choice.

Exercise 3.4: Endianness bytes

3.4.2 Transfer between different byte orders · worked solution

1. A raw byte copy preserves the sequence of bytes. The receiving system then interprets that sequence using its own byte-order rule.
2. To preserve the numeric value, the sender and receiver must agree on a serialized byte order, or one side must reorder the bytes during transfer. If bytes are copied unchanged without conversion, a multi-byte number may be read as a different value.
3. Single-byte text is unaffected. A robust file or network format specifies byte order rather than relying on the sender's native layout.

Exercise 3.4: Endianness bytes

3.4.3 Transfer 0x12345678 unchanged · worked solution

1. In little-endian order, the bytes at increasing addresses are 78, 56, 34, 12. The transfer keeps that sequence unchanged.
2. A big-endian receiver treats the first byte as most significant, so it reads 0x78563412.
3. The original value cannot be determined from those bytes alone without knowing the sender's byte order or other format metadata: the same four bytes can represent different integers under different conventions.

Made by Thanate Amornjiraporn (Focus)

Personal worked solutions based on the APDR Blackboard Exercise 3 sheet, written in my own words; no exercise-sheet prose or figures are reproduced. Not an official FAU publication; check course details against the current materials. Made by **Thanate Amornjiraporn (Focus)**.